SPRING FRAMEWORK INTERVIEW QUESTIONS

---

1. What is Spring Framework?

Spring is a lightweight, open-source Java framework used to build enterprise applications.

It provides:

- **IoC Container (Dependency Injection)**
- **AOP (Aspect Oriented Programming)**
- **Transaction Management**
- **Spring MVC**
- **Spring Boot, Spring Data, Spring Security**

Its goal is to make applications **loosely coupled, testable, modular, and maintainable**.

---

2. What is Inversion of Control (IoC) and Dependency Injection (DI)?

IoC

IoC means the **control of creating and managing objects is transferred from application code to the Spring container**.

IoC Containers

Container	Description
BeanFactory	Basic DI container, lazy initialization

Container	Description
ApplicationContext	Advanced container with AOP, events, i18n, eager loading

---

Dependency Injection (DI)

DI is the technique where Spring **injects dependencies instead of the class creating them manually**.

Types of DI

Type	Description
Constructor Injection	Best practice; ensures immutability
Setter Injection	Optional dependencies
Field Injection	Not recommended (harder for testing)

---

3. What is @Autowired? How does Spring resolve dependencies?

`@Autowired` tells Spring to automatically inject the bean.

Resolution Process

1. Match by **type**
2. If multiple → use @Primary or @Qualifier
3. If still ambiguous → Bean name
4. If still ambiguous → error: **NoUniqueBeanDefinitionException**

Behavior

- Works on **constructor**, **setter**, **fields**
- If only **one constructor exists**, Spring auto-injects *without* `@Autowired`

---

4. What happens if @Autowired is missing?

Case 1 → Single Constructor

Spring injects automatically → `@Autowired` not required.

Case 2 → Multiple Constructors

Spring throws **UnsatisfiedDependencyException**

→ You must specify with `@Autowired`.

Case 3 → Field/Setter Injection

Spring will **not inject** dependency → field remains `null`.

---

5. Difference between @Primary vs @Qualifier vs @Order

@Primary

Used when multiple beans of same type exist → marks **default bean**.

@Qualifier

Specifies exactly **which bean** should be injected.

@Order

Used **only when injecting collection of beans** or **processing beans in sequence**.

@Order is ignored for single-bean injection.

---

6. What is Circular Dependency? How does Spring handle it?

Circular dependency is when **Bean A depends on Bean B and Bean B depends on Bean A**.

Behavior with Injection Types

Injection Type	Behavior
Constructor Injection	Fails – BeanCurrentlyInCreationException
Setter Injection	Works
Field Injection	Works

Fix

- Use `@Lazy`
- Convert to setter injection
- Redesign architecture (recommended)

---

7. What is a Spring Bean? How does Spring create it?

A **bean** is an object managed by the Spring IoC container.

Ways to Create Beans

1. `@Component`, `@Service`, `@Repository`, `@Controller`
2. `@Bean` methods inside `@Configuration` class
3. XML configuration (legacy)

If you use `new` keyword?

→ Object is **not** managed by Spring.

→ No DI, no AOP, no lifecycle callbacks.

Why @Configuration uses CGLIB?

Because Spring proxies configuration classes so that:

- `@Bean` methods return **singleton bean**, not new instance
- Internal method calls also return Spring-managed beans

---

8. Difference Between JDK Dynamic Proxy vs CGLIB Proxy

Feature	JDK Dynamic Proxy	CGLIB Proxy
Works on	Interfaces	Classes
Implements?	Yes	No (creates subclass)
Performance	Medium	Fast
Default Condition	Class implements interface	No interface exists
Used for @Configuration		Always

Control using:

```
@EnableAspectJAutoProxy(proxyTargetClass = true)
```

---

9. Bean Scopes in Spring

Scope	Description
singleton	One instance per application
prototype	New bean instance on each request
request	One bean per HTTP request
session	One bean per HTTP session
application	One bean per ServletContext
websocket	One per WebSocket session

---

10. Explain Spring Bean Lifecycle (Very Important)

Full Lifecycle Sequence

1. Bean instantiation

2. Dependency injection
3. `BeanNameAware`
4. `BeanFactoryAware` or `ApplicationContextAware`
5. `BeanPostProcessor` — before init
6. Init methods:
   - `@PostConstruct`
   - `afterPropertiesSet()`
   - `initMethod`
7. `BeanPostProcessor` — after init
8. Bean ready
9. Destruction phase
   - `@PreDestroy`
   - `destroy()`
   - `destroyMethod`

---

11. Difference between @Component, @Service, @Repository, @Controller

Annotation	Layer	Purpose	Special Behavior
@Component	Generic	Marks any Spring Bean	No special behavior
@Service	Service Layer	Business logic	Semantic only
@Repository	Data Layer	DAO classes	Converts SQL exceptions to `DataAccessException`
@Controller	Web MVC	Handles UI requests	Works with DispatcherServlet
@RestController	REST APIs	Returns JSON/XML	Combines @Controller + @ResponseBody

SPRING BOOT — INTERVIEW QUESTIONS

1. What is Spring Boot? Why do we use it?

Spring Boot is a framework built on top of Spring that **simplifies application development** by providing:

- **Auto-configuration**
- **Embedded servers (Tomcat/Jetty/Netty)**
- **Opinionated defaults**
- **Starter dependencies**
- **Production-ready features (Actuator)**

Benefits

- No XML configuration
- Rapid development
- Easy deployment (jar file)
- Microservices friendly

---

2. What is Auto-Configuration in Spring Boot? How does it work?

Auto-configuration automatically configures Spring beans based on:

- Classpath dependencies
- Application properties
- Missing beans in context

It is enabled using:

```
@SpringBootApplication
```

This annotation includes:

1. `@SpringBootConfiguration`
2. `@EnableAutoConfiguration`
3. `@ComponentScan`

How auto-config works internally?

Spring Boot uses:

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

Boot loads configuration classes like `DataSourceAutoConfiguration`, `WebMvcAutoConfiguration`, etc.

Disable auto-config:

```
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
```

---

3. What are Spring Boot Starters?

Starter dependencies are pre-configured dependency sets.

Examples:

Starter	Purpose
`spring-boot-starter-web`	Build REST APIs
`spring-boot-starter-data-jpa`	JPA + Hibernate
`spring-boot-starter-security`	Security setup
`spring-boot-starter-test`	Testing tools

They simplify dependency management.

---

4. What is Spring Boot Actuator?

Actuator provides **production-ready endpoints**.

Common endpoints:

Endpoint	Description
`/actuator/health`	App health
`/actuator/metrics`	JVM, memory, CPU metrics
`/actuator/info`	App info
`/actuator/env`	Environment variables

Enable all endpoints:

```
management.endpoints.web.exposure.include=*
```

---

5. What is application.properties & application.yml in Spring Boot?

Used to configure:

- Port
- Database
- Logging
- Profiles
- Security

Example:

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost/db
```

YAML example:

```
server:
  port: 8081
spring:
  datasource:
    username: root
```

---

6. What are Spring Boot Profiles?

Profiles allow different configurations per environment.

Example:

```
spring.profiles.active=dev
```

File naming:

- `application-dev.properties`
- `application-prod.properties`

Annotation:

```
@Profile("prod")
```

---

7. Difference between Spring vs Spring Boot

Feature	Spring	Spring Boot
Config	Manual, complex	Auto-configured
Deployment	WAR	Embedded server, runnable JAR
Dependencies	Manual management	Starters simplify
Setup	Requires XML or Java config	Zero XML
Microservices	Extra config	Built-in support

---

8. How does Spring Boot run without a web.xml?

Because it uses an **embedded server** and **ServletWebServerFactory** to start Tomcat automatically.

`SpringApplication.run()` creates the complete web context without deployment to external server.

---

9. What is CommandLineRunner & ApplicationRunner?

Used to execute code at startup.

```
@Component
class StartupRunner implements CommandLineRunner {
    public void run(String... args) {
        System.out.println("App started!");
    }
}
```

SPRING AOP — INTERVIEW QUESTIONS

---

1. What is AOP (Aspect Oriented Programming)?

AOP helps apply **cross-cutting concerns** across application behavior.

Examples:

- Logging

- Transactions
- Security
- Caching

Goal: **separate cross-cutting logic from business logic.**

---

2. Key AOP Terminology

Term	Description
Aspect	Cross-cutting concern (e.g., LoggingAspect)
Advice	Code executed at join point
Join Point	Place where advice is applied (method execution)
Pointcut	Expression selecting join points
Weaving	Linking aspect with target class
Proxy	Wrapper created by Spring

---

3. Types of Advice

Type	When it runs
Before	Before method execution
After Returning	After success
After Throwing	After exception
After (finally)	Always runs
Around	Surrounds method execution (most powerful)

Example:

```
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.demo.service.*.*(..))")
```

```
    public void logBefore() {
        System.out.println("Method called");
    }
}
```

---

4. What is a Pointcut in AOP?

A pointcut expression matches the target methods.

Example:

```
@Pointcut("execution(* com.app.service.*.*(..))")
public void serviceMethods() {}
```

Then use:

```
@Before("serviceMethods()")
public void log() {}
```

---

5. What is Join Point?

A join point is a point where Spring AOP allows advising —

in Spring, join points are always method executions.

---

6. How does AOP work internally? Proxy Mechanism?

Spring AOP uses **runtime proxies**:

If class has interface → JDK Dynamic Proxy

If class has no interface → CGLIB Proxy

Proxy intercepts method calls and applies advice before/after the method.

7. Difference between Spring AOP vs AspectJ

Feature	Spring AOP	AspectJ
Type	Runtime Proxy	Compile-time
Performance	Slower	Faster
Features	Limited join points	Very powerful
Use Case	Simple enterprise apps	High-performance systems

8. Real Example: Transaction Advice

Spring uses AOP for transactions:

```
@Transactional
public void placeOrder() { ... }
```

This is implemented using AOP proxy around the method.

9. Enable AOP

```
@EnableAspectJAutoProxy
@Configuration
public class AOPConfig {}
```